Sorting and Searching, Measured

Built by AI agents; see the project repository

1 What this is

Five sorting algorithms (bubble, insertion, selection, merge and quick sort) and three searches (linear, binary and recursive binary) are written as small Python classes. Each one counts the work it does: comparisons and moves for sorting, probes for searching. Counting, unlike timing, gives the same answer on every machine, so the measurements can be checked against Big O. Every list is generated from a fixed seed. The source, the result files and the record of how the agents built it are at `https://github.com/jeremy-evert/sorting_and_searching`.

2 Sorting

Figure 1 shows comparisons against list size for random input on logarithmic axes, where a steeper line means faster growth. Bubble and selection sort lie on top of each other and grow as n^2; insertion sort is also n^2 on random input but with a smaller constant; merge and quick sort grow as $n \log n$ and stay far below.

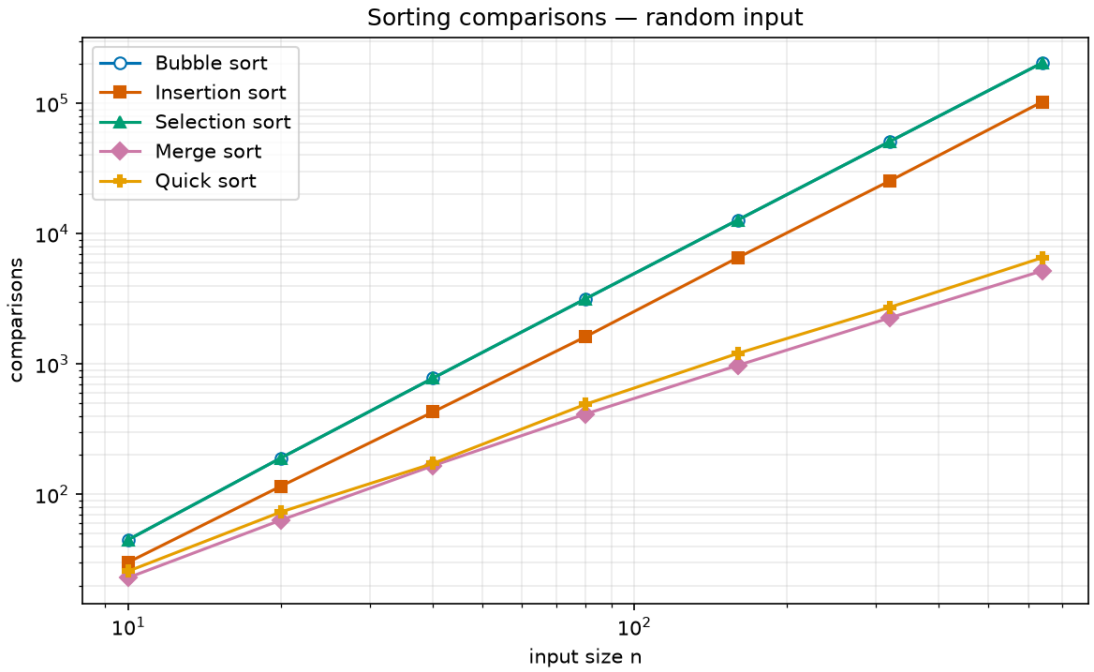


Figure 1: Comparisons against list size, random input.

3 Searching

Half of the lookups are for items in the list and half for items that are not. Linear search must look at about half the list for a hit and all of it for a miss, so on average about $3n/4$ items; binary search halves the range each probe and needs about $\log_2 n$.

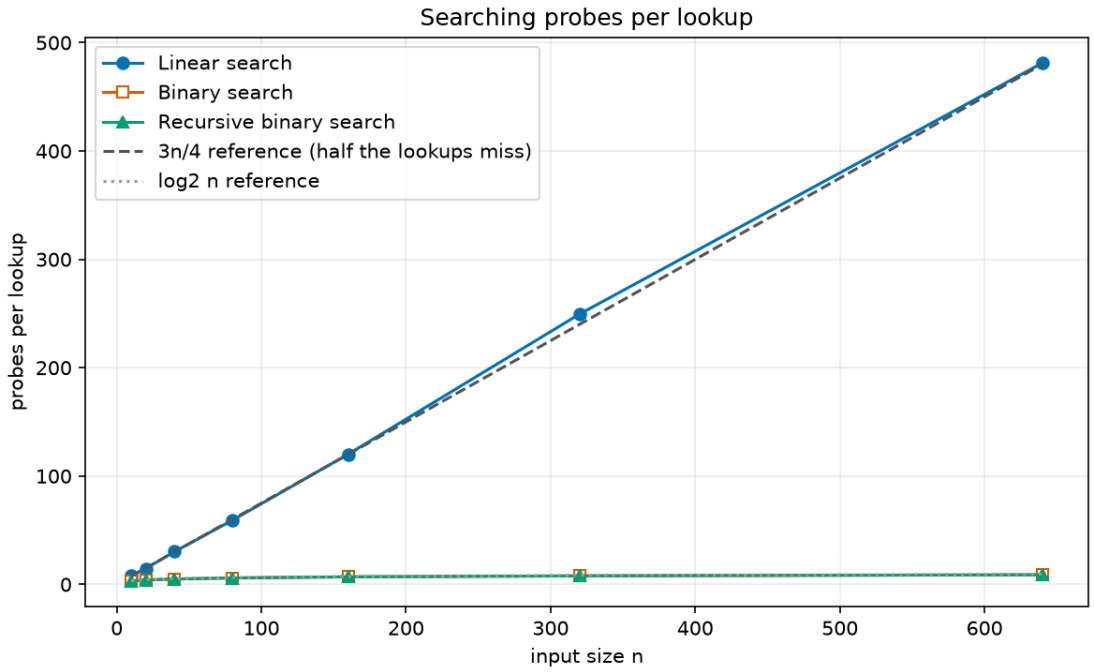


Figure 2: Probes per lookup against list size, with the $3n/4$ and $\log_2 n$ reference lines.

4 Does the theory match?

The *doubling question* asks how much more work a list twice as long costs: about $2\times$ for n, a little over $2\times$ for $n \log n$ (2.24 between sizes 320 and 640), and $4\times$ for n^2. Tables 1 and 2 compare prediction with measurement; they are the same tables as in `BIG_O.md`, which a script checks against the data. The ratio is comparisons at size 640 divided by comparisons at size 320.

5 Discussion

Most rows agree with theory. The interesting rows are the ones that appear to differ.

Insertion sort on sorted input is linear. Each item is compared once with its left neighbour and stays put: 639 comparisons at size 640.

Quick sort on sorted input is n^2. The implementation uses the first item as the pivot on purpose. On sorted input that pivot is always the smallest item, so each round removes one item and the comparison count equals bubble sort's, 204480 at size 640.

input	sorter	predicted	pred. ratio	at 320	at 640	ratio	agrees
random	bubble	n^2	4.00	51040.0	204480.0	4.01	yes
random	insertion	n^2	4.00	25451.8	102692.0	4.03	yes
random	selection	n^2	4.00	51040.0	204480.0	4.01	yes
random	merge	n log n	2.24	2259.8	5182.8	2.29	yes
random	quick	n log n	2.24	2727.6	6544.8	2.40	no
sorted	bubble	n^2	4.00	51040.0	204480.0	4.01	yes
sorted	insertion	n	2.00	319.0	639.0	2.00	yes
sorted	selection	n^2	4.00	51040.0	204480.0	4.01	yes
sorted	merge	n log n	2.24	1280.0	2880.0	2.25	yes
sorted	quick	n^2	4.00	51040.0	204480.0	4.01	yes
reversed	bubble	n^2	4.00	51040.0	204480.0	4.01	yes
reversed	insertion	n^2	4.00	51040.0	204480.0	4.01	yes
reversed	selection	n^2	4.00	51040.0	204480.0	4.01	yes
reversed	merge	n log n	2.24	1408.0	3136.0	2.23	yes
reversed	quick	n^2	4.00	51040.0	204480.0	4.01	yes

Table 1: Sorting comparisons and doubling ratios.

searcher	predicted	pred. at 320	pred. at 640	probes at 320	probes at 640
linear	3n/4	240.00	480.00	249.50	481.25
binary	log2 n	8.32	9.32	7.87	8.88
recursive binary	log2 n	8.32	9.32	7.87	8.88

Table 2: Searching probes per lookup.

Quick sort on random input shows a ratio of 2.40 against a predicted 2.24 and is marked "no" in the table. It is clearly still $n \log n$, not n^2. We tested the explanation: with 500 random lists per size the ratio is 2.31, and the exact expected count for a first-item pivot, $2(n+1)H_n - 4n$, gives 2.313. So the five-list average was unlucky, and the rest of the gap is the lower-order terms, which fade only slowly as n grows.

Merge sort has a ratio a little above 2 because doubling the list also adds a level of splitting.

Search. Linear search averages about $3n/4$ probes, not $n/2$, because misses cost the full n. Binary search sits slightly under $\log_2 n$ because hits are found before the last halving; from size 320 to 640 it rises by one probe, as a $\log n$ curve should.

In practice. At a million items, $n \log n$ is about 20 million comparisons while $n^2/2$ is about five hundred billion, roughly 25,000 times more; at a billion comparisons per second that is a fiftieth of a second against about eight minutes.